



**Universitatea
Transilvania
din Brașov**

**FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR**

Clasificarea propozițiilor relevante pentru monitorizarea sănătății folosind Machine Learning.

BRAȘOV, 2026

Introducere

Acest proiect își propune dezvoltarea unui sistem de clasificare automată a propozițiilor folosind tehnici de Machine Learning. Scopul principal este identificarea propozițiilor care conțin informații relevante pentru monitorizarea stării de sănătate și a activităților zilnice, în special în contextul unui **sistem de suport pentru persoane cu probleme de memorie sau demență**.

Datele utilizate provin din conversații de tip dialog, iar fiecare propoziție este etichetată ca fiind „importantă” sau „neimportantă”. Proiectul urmărește să construiască un model capabil să facă această distincție automat, pe baza conținutului textului.

Pentru atingerea acestui obiectiv, a fost implementat un pipeline complet de Machine Learning, care include analiza datelor, preprocesarea textului, antrenarea mai multor modele și evaluarea performanței acestora. O atenție deosebită a fost acordată dezechilibrului inițial al dataset-ului, care a fost corectat prin tehnici de echilibrare, pentru a obține rezultate relevante și corecte.

Rezultatul final este un model eficient, capabil să clasifice propozițiile cu o acuratețe ridicată, demonstrând aplicabilitatea metodelor de Machine Learning în procesarea limbajului natural.

Generarea și selectarea dataset-ului

În cadrul proiectului au fost utilizate două abordări diferite pentru obținerea datelor.

În etapa inițială, a fost creat un dataset propriu, **generat artificial folosind șabloane și reguli**. Acest dataset conținea propoziții etichetate manual ca fiind „importante” sau „neimportante”, pe baza unor cuvinte cheie și a unor tipare predefinite. Scopul acestei

etapa a fost de a înțelege mai bine problema și de a testa rapid pipeline-ul de Machine Learning.

```
# Seed examples
# These original sentences are always included in the output.
seed_examples = [
    ("I took my medicine in the morning", 1),
    ("I went to the pharmacy", 1),
    ("I had lunch with my daughter", 1),
    ("I visited the doctor", 1),
    ("I paid my bills online", 1),
    ("I watched television for two hours", 0),
    ("I slept in the afternoon", 0),
    ("I scrolled on my phone", 0),
    ("I played a video game", 0),
    ("I read a magazine", 0),
]

# Templates for generating more examples.
time_phrases = [
    "in the morning",
    "in the afternoon",
    "in the evening",
    "before bed",
    "after breakfast",
    "after dinner",
    "during the day",
    "at night",
]

duration_phrases = [
    "for a few minutes",
    "for an hour",
    "for a few days",
    "for a few weeks",
    "for a few months",
    "for a few years",
]
```

```
health_templates = [
    "I had severe {symptom} and needed medical attention {time}",
    "I forgot my {item} and my condition got worse {time}",
    "I set a reminder to take my {item} {time}",
    "I scheduled an appointment with the {professional} {time}",
    "I checked the expiry date on my {item} {time}",
    "I called the {professional} to confirm my visit {time}",
    "I went to the {place} for a checkup {time}",
    "I made sure to drink enough water today {time}",
    "I measured my blood pressure in the {place} {time}",
    "I asked my family to remind me about the {task} {time}",
    "I wore my {device} as the doctor advised {time}",
    "I recorded my symptoms in a notebook {time}",
    "I followed my {task} plan {time}",
    "I reviewed my medication list {time}",
    "I used my {device} to get to the kitchen {time}",
    "{caregiver} helped me with my {task} {time}",
    "I felt some {symptom} so I rested {time}",
    "I applied the {item} to my skin {time}",
    "I practiced my {task} with the {professional} {time}",
    "I brushed my teeth and did my hygiene routine {time}",
    "I took a shower with help from {caregiver} {time}",
    "I updated {caregiver} about my {symptom} {time}",
    "I organized my {item} into the weekly pill box {time}",
    "I checked my calendar for the next {professional} visit {time}",
    "I set up my {device} to remind me about the {task} {time}",
    "I made sure to get some exercise as part of my {task} {time}",
    "I followed the doctor's advice and took my {item} {time}",
    , etc
```

```

# We add the seed examples first, then generate more until we hit the target.
# 'seen' tracks which (sentence, label) pairs have already been added.
target_count = 50000

seen = set()
unique_examples = []

for sentence, label in seed_examples:
    key = (sentence.strip().lower(), label)
    if key not in seen:
        seen.add(key)
        unique_examples.append((sentence, label))

# Try to generate enough unique examples; If we make too many attempts without
# increasing the dataset size, we stop to avoid an infinite loop
max_attempts = target_count * 50
attempts = 0

# Keep the dataset balanced by generating (roughly) the same number of examples per class.
target_per_class = target_count // 2
count_0 = 0
count_1 = 0

while len(unique_examples) < target_count and attempts < max_attempts:
    attempts += 1

    # Choose the next label so we end up with roughly balanced classes.
    if count_1 < target_per_class:
        label = 1
    elif count_0 < target_per_class:
        label = 0
    else:
        # We've filled both classes to the desired size.
        break

    if label == 1:
        # For health/self-care examples, fill a template with random words.
        template = random.choice(health_templates)
        time = random.choice(time_phrases)

```

```

        template = random.choice(other_templates)
        if random.random() < 0.5:
            time = random.choice(time_phrases)
        else:
            time = ""
        sentence = template.format(
            time=time,
            duration=random.choice(duration_phrases),
            hobby=random.choice(hobbies),
            item=random.choice(household_items),
            friend=random.choice(friends),
            activity=random.choice(activities),
            electronic=random.choice(electronics),
            chore=random.choice(chores),
        )

        key = (sentence.strip().lower(), label)
        if key not in seen:
            seen.add(key)
            unique_examples.append((sentence, label))
            if label == 1:
                count_1 += 1
            else:
                count_0 += 1

    if len(unique_examples) < target_count:
        # Print a warning if we could not build enough unique rows.
        print(
            "Warning: could not generate enough unique examples; "
            f"generated {len(unique_examples)} unique rows after {attempts} attempts."
        )

# Write the final dataset out to 'data/dataset.csv'.
with open(DATA_PATH, "w", encoding="utf-8") as f:
    f.write("sentence,label\n")
    for sentence, label in unique_examples:
        f.write(f"{sentence},{label}\n")

print(f"Wrote {len(unique_examples)} rows to {DATA_PATH}")

```

Etichetarea dataset-ului (reguli și LLM)

În cadrul proiectului au fost utilizate două metode de etichetare a datelor: o metodă bazată pe reguli și o metodă bazată pe un model de tip LLM. Aceste două abordări au fost folosite complementar pentru a obține atât un volum mare de date, cât și o calitate mai bună a etichetelor.

Etichetarea bazată pe reguli:

Prima metodă implementată a fost una bazată pe reguli și cuvinte cheie. În acest scop, au fost definite mai multe liste de termeni relevante pentru domeniul sănătății, cum ar fi simptome, tratamente, situații de risc sau activități importante. Pentru fiecare propoziție, s-a calculat un scor folosind funcția `compute_score`, care verifică prezența acestor cuvinte cheie și atribuie puncte în funcție de importanța lor. De exemplu, prezența unor termeni legați de simptome sau tratamente adaugă un scor mai mare, în timp ce alte categorii contribuie mai puțin.

```
def compute_score(sentence: str) -> int:
    s = sentence.lower()
    score = 0

    # High importance (health + symptoms)
    if any(word in s for word in health_keywords):
        score += 2

    if any(word in s for word in symptom_keywords):
        score += 2

    # Medium importance (treatment + risk)
    if any(word in s for word in treatment_keywords):
        score += 2

    if any(word in s for word in risk_keywords):
        score += 2

    # Medium-low (daily + care + monitoring)
    if any(word in s for word in daily_important_keywords):
        score += 1

    if any(word in s for word in care_keywords):
        score += 1

    if any(word in s for word in monitoring_keywords):
        score += 1

    if "feel" in s or "felt" in s:
        score += 1

    if "help" in s:
        score += 1

    return score
```

Pe baza acestui scor, funcția `label_sentence` atribuie o etichetă:

- 1 pentru propoziții considerate importante
- 0 pentru propoziții neimportante

```
✓ def label_sentence(sentence: str) -> int:
    score = compute_score(sentence)

    return 1 if score >= 1 else 0
```

Această metodă permite etichetarea rapidă a unui volum mare de date, însă are limitări deoarece se bazează pe reguli fixe și nu poate înțelege contextul propozițiilor.

Echilibrarea dataset-ului:

După etichetare, s-a observat un dezechilibru între clase. Pentru a corecta acest lucru, s-a aplicat tehnica de oversampling asupra clasei minoritare, duplicând aleator exemplele acesteia până la egalarea cu clasa majoritară. Această etapă a fost necesară pentru a preveni biasul modelului și pentru a permite învățarea corectă a ambelor clase.

Etichetarea folosind LLM

A doua metodă utilizează un model de tip LLM pentru etichetarea propozițiilor. În acest caz, fiecare propoziție este trimisă către model împreună cu un prompt care descrie clar criteriile de clasificare și oferă exemple.

Modelul returnează un răspuns de tip YES sau NO, care este apoi convertit în etichete numerice (1 sau 0). Pentru a limita costurile și timpul de execuție, această metodă a fost aplicată doar pe un subset de date.

```
client = OpenAI()

# LLM LABEL FUNCTION
✓ def label_with_llm(sentence: str) -> int:
    prompt = f"""
    You are labeling sentences for a machine learning dataset used in a dementia support system.

    Goal:
    Detect whether a sentence contains IMPORTANT information that should be tracked to help monitor a person's daily health, safety, memory, or essential functioning.

    A sentence is IMPORTANT (YES) if it includes ANY of the following:
    - Health-related actions (medicine, doctor visits, treatment)
    - Symptoms or physical/mental condition (pain, dizziness, confusion, feeling unwell)
    - Memory issues (forgetting, missing things, confusion)
    - Safety risks (falling, injury, emergency, needing help)
    - Essential daily responsibilities (appointments, schedules, reminders)
    - Monitoring health (blood pressure, glucose, vital signs)
    - Assistance from others (caregiver help, needing support)

    A sentence is NOT IMPORTANT (NO) if it is:
    - Casual conversation (greetings, small talk)
    - Entertainment or hobbies (TV, music, games)
    - General opinions or vague statements without health/safety relevance
    - Neutral daily actions without risk or importance (e.g., "I went for a walk")

    Be strict: only label YES if the sentence clearly contributes to tracking health, safety, or critical daily functioning.

    Examples:
    "I took my medicine this morning" → YES
    "I feel dizzy and weak today" → YES
    "I forgot my appointment again" → YES
    "I fell in the kitchen" → YES
    "My daughter helped me take my pills" → YES

    "I watched TV last night" → NO
    "I like this song" → NO
    "I went outside for a walk" → NO
    """
```

```

Don't count on it .,0,0
"Yes , what kind of smell do you like ?",0,0
They're really running you into the ground . Why don't you ask for some time off ? You could take a long weekend and go away somewhere .,0,0
"Yes , thanks . By the way , what time do I have to start check-in ?",0,0
Some grape juice would also be nice .,0,0
Yes . This is Janet Smith . I'd like to make an appointment with Dr . Mustn as soon as possible .,1,1
"Sorry , I'm very near sighted .",0,0
I'm tired . I need a rest .,1,0
"Hey , I've always gone hunting or fishing on vacation . I am sorry that bothers you .",0,0
It's too bad that you have to leave so early .,0,0
"Will this do ? Your major is not related to banking , as you know .",1,0
My pleasure . You're a sweet couple . Do you have any questions ?,0,0
"Don't be too sad . If you really think that you have no feeling with him , then , in my opinion , getting divorced maybe is the best way to solve the problem .",0,0
Yes . I'd like to use a safety deposit box .,0,0
Las Vegas is in the southwest .,0,0

```

Ulterior, pentru a obține un set de date mai realist și mai variat, s-a utilizat un dataset existent, **DailyDialog**. Acesta conține conversații naturale, apropiate de limbajul real folosit în viața de zi cu zi. Pe acest dataset au fost aplicate metode de etichetare automată, folosind reguli bazate pe cuvinte cheie și, în unele cazuri, modele de tip LLM pentru generarea etichetelor.

De ce s-a făcut această tranziție:

Dataset-ul generat manual a fost util pentru testare, dar avea limitări:

- variație redusă a limbajului
- expresii repetitive
- lipsa realismului

Prin utilizarea dataset-ului DailyDialog:

- s-a obținut un limbaj mai natural
- modelul a fost expus la exemple mai diverse
- performanța a devenit mai relevantă pentru aplicații reale

Etapele descrise:

1. ANALIZA EXPLORATORIE A DATELOR (EDA)

În această etapă a fost creat un script separat (eda.py) care analizează dataset-ul înainte de antrenarea modelului.

Au fost calculate următoarele:

- numărul total de exemple:

```
print("Number of samples:", len(data))
```

- distribuția claselor:

```
print("\nClass distribution:")
#value_counts() method is used to count the occurrences of each unique value in the "label" column
#it represents the class distribution in the dataset.
print(data["label"].value_counts())
```

- lungimea propozițiilor:

```
#calculating the length of each sentence and storing it in a new column called "length".
data["length"] = data["sentence"].apply(lambda x: len(x.split()))

print("\nAverage sentence length:", data["length"].mean())
```

- frecvența cuvintelor:

```
# split each preprocessed sentence into words and flatten into a single list
tokens = [w for sent in cleaned for w in sent.split()]
counts = Counter(tokens)
```

Scopul EDA este să înțelegem:

- dacă dataset-ul este echilibrat
- dacă textul conține informații relevante
- dacă există probleme evidente (zgomot, lipsă de semnal)

Fără acest pas, modelul ar fi antrenat în orb, fără să știm dacă datele sunt potrivite.

Ce s-a descoperit:

Rezultatele au arătat (pentru datasetul non-generat):

- dataset puternic dezechilibrat
- aproximativ 90% clasa 0 și 10% clasa 1
- cele mai frecvente cuvinte erau: yes, im, like, well; acestea sunt specifice conversațiilor casual
- cuvintele relevante medical: medicine, doctor; apăreau foarte rar

```
P5 C:\School\LS\proiect_ml_dementia> & C:/Users/Ana/AppData/Local/Programs/Python/Python31
label
0    909
1     91
Name: count, dtype: int64

Average sentence length: 13.988

Top 20 words after normalization:
      yes  78
      im  71
     like  67
     well  48
     dont  48
    would  46
      go  46
     see  41
  really  40
     good  38
  please  38
     know  38
     take  37
     want  36
    think  35
     time  34
      oh  33
      one  32
    going  31
     need  30

Important keywords (should be relevant): {'medicine': 0, 'doctor': 5}
Unimportant keywords (might be noise): {'tv': 3, 'sleep': 0}
```


Interpretare:

Aceste rezultate indică două probleme majore:

- Dezechilibru de clase
- Modelul ar tinde să prezică doar clasa majoritară
- Semnal slab pentru clasa importantă
- Modelul nu are suficiente indicii pentru a învăța diferența

Decizia luată pe baza EDA:

Pe baza acestei analize, s-au făcut două alegeri importante:

- aplicarea unei tehnici de echilibrare a datelor (oversampling; se mărește artificial clasa minoritară pentru a o aduce la același nivel cu clasa majoritară.)
- folosirea unor metode de preprocesare pentru a evidenția cuvintele relevante

Concluzie pentru această etapă:

EDA a fost esențială pentru identificarea problemelor dataset-ului.

Fără acest pas, modelul ar fi avut performanțe înșelătoare, cu accuracy mare dar inutil în practică.

2. CURĂȚAREA ȘI PREPROCESAREA DATELOR

Ce s-a implementat

Această etapă a fost implementată într-un fișier separat (preprocessing.py) și are două funcții principale:

- `clean_data_simple()`
- `preprocess_text()`

1. Curățarea datelor

În `clean_data_simple()` s-au realizat:

```
def clean_data_simple(data):

    print("Initial dataset size:", len(data))

    clean_rows = []
    removed_missing = []

    for _, row in data.iterrows(): #iterates over each row in the dataset using iterrows()
        if isinstance(row["sentence"], str):
            clean_rows.append(row)
        else:
            removed_missing.append(row) #if missing remove

    print("\nMissing values removed:", len(removed_missing))
    for r in removed_missing[:3]: #prints the first 3 removed rows with missing values for inspection
        print("Removed (missing):", r["sentence"])

```

-eliminarea valorilor lipsă (doar propoziții valide sunt păstrate)

Inițial s-a implementat și eliminarea duplicatelor, dar ulterior această logică a fost eliminată deoarece afecta negativ oversampling-ul.

```
for row in clean_rows:
    sentence = row["sentence"]

    # if sentence not in seen: #checks if the sentence has already been seen (to remove duplicates)
    #     seen.add(sentence)
    final_rows.append(row)
    # else:
    #     removed_duplicates.append(row)

print("\nDuplicate rows removed:", len(removed_duplicates))
for r in removed_duplicates[:3]:
    print("Removed (duplicate):", r["sentence"])

print("\nFinal dataset size:", len(final_rows))

return pd.DataFrame(final_rows) #returns a new DataFrame containing only the cleaned rows

```

2. Preprocesarea textului

În preprocess_text():

```
#preprocess_text function: lowercases, removes punctuation, and removes stopwords from the input
def preprocess_text(text):

    text = text.lower()

    text = re.sub(r"[^\w\s]", "", text) #any character that is not a word character and not whitespace
    words = text.split() #splits into words (tokens)

    words = [w for w in words if w not in stop_words]

    return " ".join(words) #joins the remaining words back into a single string and returns it

```

- transformare în lowercase

- eliminare semne de punctuație
- tokenizare simplă
- eliminare stopwords folosind NLTK
- reconstruirea textului

Preprocesarea datelor a fost separată de etapa de feature extraction. Curățarea textului (lowercase, eliminare punctuație și stopwords) a fost realizată într-un modul dedicat, iar transformarea în vectori numerici s-a realizat ulterior folosind TF-IDF în etapa de antrenare a modelului.

```
#using the TfidfVectorizer to convert text data into numerical features
vectorizer = TfidfVectorizer() You, 38 minutes ago • Tweaking the parameters for diffi
#for counting words and weighing them based on their importance in the dataset

X = vectorizer.fit_transform(data["clean"])
#X is obtained by vectorizing the "clean" column of the dataset, then the dataset is split into
```

Modelele de Machine Learning nu pot lucra direct cu text, deci este necesar:

- să eliminăm zgomotul (punctuație, stopwords)
- să standardizăm textul
- să convertim textul în valori numerice

TF-IDF a fost ales deoarece:

- evidențiază cuvintele importante
- reduce impactul cuvintelor frecvente dar irelevante
- Stopwords au fost eliminate pentru a reduce zgomotul
- Duplicatele NU au fost eliminate în final, pentru a nu anula oversampling-ul

Impact asupra modelului:

După preprocesare:

- datele devin mai consistente
- modelul poate învăța mai ușor tipare relevante
- performanța crește față de date brute

Concluzie

Această etapă este esențială deoarece face legătura între text brut și model.

Fără preprocesare, modelul ar învăța din date zgomotoase și ar avea rezultate slabe.

3. ÎMPĂRȚIREA DATELOR ÎN SETURI DE ANTRENAMENT ȘI TESTARE

După preprocesare și transformarea textului în vectori TF-IDF, datele au fost împărțite în două seturi:

```
X = vectorizer.fit_transform(data["clean"])
#X is obtained by vectorizing the "clean" column of the dataset, then the dataset is split

y = data["label"]
#splitting the dataset into training and testing sets, with 80% for training and 20% for te
```

Apoi:

```
#splitting the dataset into training and testing sets, with 80% for training and 20% for
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
#random state is set to 42 to ensure reproducibility of the results
```

Ce înseamnă această împărțire:

- X_{train} , y_{train} → folosite pentru antrenarea modelului
- X_{test} , y_{test} → folosite pentru evaluare

Raportul:

80% training

20% test

Impact asupra modelului:

Această etapă permite:

- evaluare corectă și realistă
- compararea consistentă între modele
- evitarea biasului în rezultate

Concluzie:

Împărțirea datelor este un pas esențial pentru validarea modelului.

4. CREAREA ȘI ANTRENAREA MODELELOR

În această etapă au fost definite și testate mai multe modele de clasificare.

Modelele utilizate au fost:

- Logistic Regression
- Naive Bayes

- Support Vector Machine

Acestea au fost definite într-un dicționar:

```
# Define models to train
models = {
    # Logistic Regression
    "LogReg_C1": LogisticRegression(C=1, max_iter=1000, class_weight="balanced"),
    "LogReg_C0.1": LogisticRegression(C=0.1, max_iter=1000, class_weight="balanced"),

    # Naive Bayes
    "NB_alpha1": MultinomialNB(alpha=1.0),
    "NB_alpha0.5": MultinomialNB(alpha=0.5),

    # SVM
    "SVM_C1": LinearSVC(C=1, class_weight="balanced"),
    "SVM_C0.5": LinearSVC(C=0.5, class_weight="balanced")
}
```

Fiecare model a fost antrenat folosind:

```
for name, model in models.items():
    model.fit(X_train, y_train)
```

și evaluat pe setul de test:

```
predictions = model.predict(X_test)
```

De ce au fost alese aceste modele:

Aceste modele sunt potrivite pentru clasificarea textului deoarece:

- funcționează bine cu date sparse (cum sunt vectorii TF-IDF)
- sunt eficiente și rapide
- sunt ușor de interpretat

De ce s-au testat mai mulți parametri:

Pentru fiecare model s-au modificat hiperparametri:

1. Logistic Regression

C controlează cât de mult penalizează modelul erorile

- C mic → model mai simplu
- C mare → model mai flexibil

2. Naive Bayes

alpha controlează smoothing-ul

- previne probabilități zero
- ajută la generalizare

3. SVM

C controlează compromis-ul între:

- clasificare corectă
- generalizare

Cum s-a făcut compararea:

Toate modelele au fost evaluate automat într-un loop:

```
for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    report = classification_report(y_test, predictions, output_dict=True)

    results.append({
        "model": name,
        "accuracy": report["accuracy"],
        "f1": report["weighted avg"]["f1-score"]
    })

# f1-score
results = sorted(results, key=lambda x: x["f1"], reverse=True)

print("\n Best Models:")
for r in results:
    print(r)
```

Rezultatele au fost salvate și sortate după F1-score.

```
PS C:\School\LS\proiect_ml_dementia> & C:/Users/Ana/AppData/Local/Programs/Python/Python312/python.exe c:/School/LS/proiect_ml_dementia/src/train_model.py
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Ana\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
Initial dataset size: 161314

Missing values removed: 0

Duplicate rows removed: 0

Final dataset size: 161314

Best Models:
{'model': 'SVM_C1', 'accuracy': 0.9299507175402163, 'f1': 0.9299349552793607}
{'model': 'SVM_C0.5', 'accuracy': 0.9281529925921334, 'f1': 0.9281252460808801}
{'model': 'LogReg_C1', 'accuracy': 0.9036047484734836, 'f1': 0.9034358629684961}
{'model': 'NB_alpha0.5', 'accuracy': 0.8596534730186282, 'f1': 0.8595379955940907}
{'model': 'NB_alpha1', 'accuracy': 0.8566779282769736, 'f1': 0.8565278123351691}
{'model': 'LogReg_C0.1', 'accuracy': 0.8550661748752441, 'f1': 0.8540452374880496}
PS C:\School\LS\proiect_ml_dementia>
```

Ce s-a observat:

- SVM a avut cele mai bune rezultate
- Logistic Regression a avut performanță apropiată

- Naive Bayes a fost mai slab

Motiv:

- SVM gestionează foarte bine datele de tip TF-IDF
- Naive Bayes face presupuneri simple care nu se potrivesc perfect cu datele

Concluzie:

Această etapă a permis compararea mai multor modele și alegerea unei configurații optime.

Testarea mai multor hiperparametri a fost esențială pentru obținerea unei performanțe ridicate.

5. CONSTRUIREA MODELULUI FINAL

După testarea tuturor modelelor și configurațiilor, s-a ales modelul final pe baza performanței obținute pe setul de test.

Rezultatele au fost comparate folosind în principal F1-score, deoarece oferă un echilibru între precision și recall.

Pe baza rezultatelor obținute:

- SVM_C1 $\rightarrow$ F1 $\approx$ 0.93

Modelul ales a fost:

Support Vector Machine cu $C = 1$

De ce s-a ales acest model:

Motivul principal a fost:

- cel mai mare F1-score
- performanță stabilă
- echilibru bun între clase

De ce nu s-au ales celelalte modele:

1. Logistic Regression
 - rezultate apropiate, dar ușor mai slabe
2. Naive Bayes

- performanță mai scăzută
3. În SVM:
- C controlează cât de mult modelul încearcă să clasifice corect toate exemplele

În cazul dat:

- $C = 1$ a oferit cel mai bun echilibru
- valori mai mici au dus la subantrenare
- valori mai mari nu au adus îmbunătățiri semnificative

Ce înseamnă rezultatul obținut: $F1 \approx 0.93$

Modelul:

- clasifică corect majoritatea exemplilor
- nu favorizează o clasă
- are performanță consistentă

Decizie finală:

Modelul SVM cu $C = 1$ a fost ales ca model final și va fi folosit pentru evaluarea finală și eventuale aplicații.

Concluzie:

Această etapă a fost esențială pentru selectarea celei mai bune configurații de model. Alegerea s-a bazat pe comparații obiective și pe metrice relevante pentru problemă.

6. TESTAREA ȘI VALIDAREA MODELULUI

După alegerea modelului final, acesta a fost evaluat pe setul de test folosind:

```
15 from sklearn.metrics import classification_report  
report = classification_report(y_test, predictions, output_dict=True)
```

Această funcție calculează automat următoarele metrice:

- accuracy
- precision
- recall

- F1-score

Evaluarea se face pe date nevăzute în timpul antrenării, ceea ce oferă o estimare realistă a performanței.

Ce înseamnă fiecare metrică:

1. **Accuracy:** Procentul total de predicții corecte.
în cazul dat: ≈ 0.93
2. **Precision:** Din toate predicțiile pozitive (clasa 1), câte sunt corecte.
răspunde la: „Când modelul spune că ceva este important, cât de des are dreptate?”
3. **Recall:** Din toate exemplele reale din clasa 1, câte au fost detectate.
răspunde la: „Câte cazuri importante reușește modelul să găsească?”
4. **F1-score:** Media armonică între precision și recall.
cea mai importantă metrică în acest proiect

De ce s-a ales F1-score:

Inițial, dataset-ul era dezechilibrat, deci:

- accuracy putea fi înșelător
- modelul putea ignora complet clasa 1

F1-score oferă:

- evaluare echilibrată
- penalizează modelele care ignoră o clasă

Ce s-a observat în rezultate:

După corectarea pipeline-ului:

- accuracy $\approx$ F1 ≈ 0.93
- nu mai apar diferențe mari între metrici
- modelul clasifică bine ambele clase

Confirmă că:

- dataset-ul este echilibrat
- modelul nu este biased

Probleme întâlnite și soluții:

Problemă: Modelul ignora clasa minoritară și apăreau warning-uri

Cauză: Dezechilibru în dataset

Soluție: Aplicarea tehnicii de oversampling

Problemă: Oversampling-ul nu avea efect

Cauză: Eliminarea duplicatelor după balansare

Soluție: Eliminarea acelei etape din preprocessing

Concluzie:

Etapa de testare confirmă că modelul ales este eficient și robust.

Folosirea metricilor corecte și corectarea problemelor din date au fost esențiale pentru obținerea unui rezultat relevant.

Concluzie generală

Proiectul a implementat complet un pipeline de Machine Learning pentru clasificarea textului, incluzând toate etapele necesare: analiză exploratorie, preprocesare, antrenare, selecție și evaluare. Rezultatele obținute arată că modelul este capabil să distingă corect între propoziții importante și neimportante, iar tehnicile aplicate, în special echilibrarea datelor, au avut un impact major asupra performanței.